

---

# TRISPI

## Technical Whitepaper

---

### **The AI-Powered, Post-Quantum Secure, Web4 Blockchain Protocol**

Version 3.0 | May 2026

*Proof of Intelligence (PoI) | Federated Learning v2 | Ed25519 + Dilithium3 PQC | Validator Registry | Energy Provider Rewards*

---

TRISPI Network | <https://github.com/TRISPIAINETWORK/TRISPI>

This document is for technical reference only. No financial or investment advice.

## Table of Contents

---

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>1 Abstract</b>                                          | <b>3</b>  |
| <b>2 Introduction -- Web4 Vision and Problem Statement</b> | <b>3</b>  |
| <b>3 System Architecture</b>                               | <b>5</b>  |
| <b>4 Proof of Intelligence (Pol) Consensus</b>             | <b>7</b>  |
| <b>5 Federated Learning v2</b>                             | <b>10</b> |
| <b>6 Post-Quantum Cryptography</b>                         | <b>12</b> |
| <b>7 Dual Runtime -- EVM and WASM</b>                      | <b>13</b> |
| <b>8 P2P Network Layer</b>                                 | <b>14</b> |
| <b>9 IBC Multi-Chain and Parachain Registry</b>            | <b>15</b> |
| <b>10 TRP Token Economics</b>                              | <b>16</b> |
| <b>11 Standalone Energy Provider</b>                       | <b>17</b> |
| <b>12 Security Model</b>                                   | <b>20</b> |
| <b>13 Current Challenges and Solutions</b>                 | <b>22</b> |
| <b>14 Conclusion and Roadmap</b>                           | <b>25</b> |
| A Full API Reference                                       | 26        |
| B genesis.json Schema                                      | 28        |
| References                                                 | 29        |
| C Glossary                                                 | 30        |

# 1. Abstract

---

TRISPI (Trustless Reasoning and Intelligence System for Programmable Infrastructure) is a Web4 blockchain network that replaces energy-wasteful proof-of-work with Proof of Intelligence (Pol) -- a consensus mechanism in which AI agents evaluate and validate transactions using a custom neural network trained continuously through a decentralized Federated Learning v2 protocol. Version 3.0 introduces four major advances over the original protocol: (1) a permissionless Validator Registry where any external node may register an Ed25519 public key, submit signed Pol scores, and earn TRP rewards weighted by a continuously-updated trust score; (2) a fully decentralised Federated Learning pipeline with AES-256-GCM encrypted gradient submission and on-chain aggregation commitment hashes that anyone can verify; (3) a standalone Energy Provider client (`trispi_energy_provider.py`) that any machine can run to begin earning TRP through three parallel reward streams; and (4) a distributed transaction-fraud detection system in which external validators submit signed fraud verdicts that are weighted by their trust score before consensus.

The network achieves post-quantum security through a hybrid Ed25519 + Dilithium3 (ML-DSA) signature scheme and Kyber1024 key exchange, aligned with NIST FIPS 203 and FIPS 204. A dual EVM + WASM smart contract runtime enables Solidity contracts and CosmWasm modules to coexist on the same chain. The TRP token implements EIP-1559 fee mechanics with 70% base-fee burn and Bitcoin-style halving every 210,000 blocks, targeting a 50 million token hard cap.

This document is intended for engineers, researchers, and node operators. No investment or financial information is presented.

# 2. Introduction -- Web4 Vision and Problem Statement

---

## 2.1 The Limitations of Existing Blockchains

First-generation blockchains consume extraordinary electrical energy performing proof-of-work calculations that produce no externally useful output. Proof-of-stake systems improve energy efficiency but introduce plutocratic validator selection where influence is proportional to capital rather than capability. Neither paradigm harnesses the latent compute capacity of the global internet in a way that creates genuine value outside the chain itself.

Simultaneously, the cryptographic foundations of existing blockchains -- ECDSA and RSA -- are vulnerable to Shor's algorithm on a sufficiently powerful quantum computer. The 'harvest now, decrypt later' attack vector means that transactions signed today with classical keys can be decrypted retroactively once quantum hardware matures, threatening the long-term security of stored value.

## 2.2 The Web4 Model

TRISPI defines Web4 as the convergence of three properties: (1) AI-native infrastructure where machine intelligence participates in consensus rather than merely running on top of it; (2) quantum-resistant security so that long-term value storage is safe against foreseeable cryptanalytic advances; and (3) composable multi-chain architecture where application-specific blockchains interoperate through a lightweight IBC-compatible protocol without surrendering sovereignty over their own validator sets.

## 2.3 What Is New in v3.0

### Permissionless Validator Registry

Any node may register an Ed25519 public key via POST /api/validators/register and begin submitting signed PoI block scores and fraud verdicts. Trust weights are maintained on-chain and updated automatically per round.

### Federated Learning v2

Gradient submission is now end-to-end encrypted with AES-256-GCM using a per-provider per-round key derived from the provider's Ed25519 public key and Kyber1024. After each aggregation round, a SHA3-256 commitment hash is posted on-chain and verifiable via GET /api/federated/verify-round/{round\_id}.

### Three Earning Loops

Energy Providers earn TRP through three independent parallel loops: 0.1 TRP per scored block (every 15 s), 1.0 TRP per accepted FL gradient (every 30 s), and 0.01 TRP per fraud verdict (every 5 s).

### Fraud Detection v1

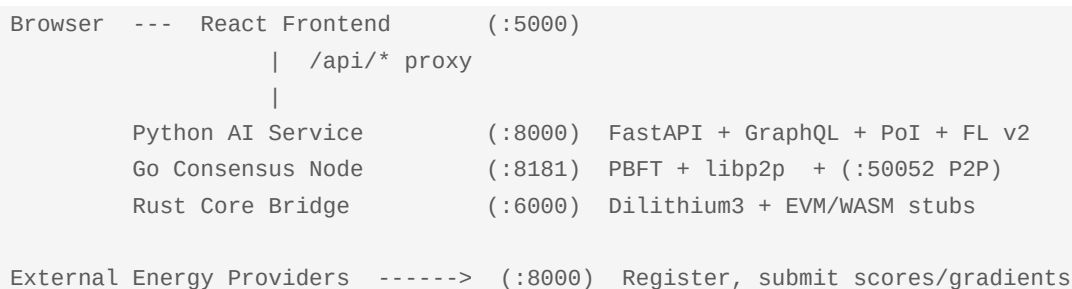
A 5-feature numpy MLP evaluates every submitted transaction. External validators submit signed verdicts; weighted consensus (>60% must agree) determines the final classification.

### Go Consensus Port Update

The Go consensus node now listens on port 8181 (HTTP) and 50052 (libp2p P2P), replacing the previous ports 8081 and 50051 respectively.

### 3. System Architecture

TRISPI v3.0 is a four-service architecture. Each service is independently deployable and communicates over well-defined HTTP interfaces. External Energy Providers connect from any internet-reachable machine.



#### 3.1 Python AI Service (Port 8000)

The Python AI service is the primary entry point for all external clients. It is a FastAPI application that implements: the full REST API consumed by the frontend and external providers; a GraphQL API backed by strawberry-graphql with WebSocket subscriptions for real-time block feeds; the PoI consensus engine; the Federated Learning v2 coordinator; the Validator Registry; the fraud detection pipeline; the EIP-1559 gas market; wallet creation and balance tracking; smart contract deployment and invocation; and the multi-chain parachain registry.

Three middleware layers are applied in order: AISecurityMiddleware (IP trust scoring, signature verification, attack logging), RateLimitMiddleware (sliding-window rate limiting, configurable per deployment), and SecurityHeadersMiddleware (CSP, X-Frame-Options, HSTS, and others). The service loads 1,055 genesis balances at startup and initialises all AI models -- startup takes approximately 30 seconds on first run.

#### 3.2 Go Consensus Node (Port 8181)

The Go consensus node runs the PBFT agreement protocol and the libp2p peer-to-peer layer. It exposes an HTTP API on port 8181 and accepts libp2p connections on port 50052. Key API endpoints: POST /api/p2p/announce (peer self-announcement), GET /api/p2p/bootstrap (node coordinates and chain head), GET /api/p2p/peers (peer list), and the PBFT message handlers (HandlePBFTPrePrepare, HandlePBFTPrepare, HandlePBFTCommit in p2p.go). Peer reputation and rate-limiting are managed by peer\_scoring.go.

The Go node is the authority on canonical chain state. When the Go node is unavailable, the Python service continues to produce blocks in standalone mode -- the intended single-node deployment scenario.

#### 3.3 Rust Core Bridge (Port 6000)

The Rust core bridge provides real Dilithium3 (ML-DSA) signing and verification using the pqcrypto-dilithium 0.5 crate (NIST PQC reference implementation), approximately 40x faster than the dilithium-py pure-Python fallback. It also provides thin EVM and WASM execution stubs.

| Method | Path    | Description                                 |
|--------|---------|---------------------------------------------|
| GET    | /health | Node health, version, Dilithium3 capability |
| GET    | /pubkey | Node Dilithium3 public key (hex)            |

|      |            |                                                |
|------|------------|------------------------------------------------|
| GET  | /stats     | Request counters (signs_total, verifies_total) |
| POST | /sign      | Sign hex-encoded payload with Dilithium3       |
| POST | /verify    | Verify a Dilithium3 detached signature         |
| POST | /evm/exec  | EVM execution stub (gas estimate, stub=true)   |
| POST | /wasm/exec | WASM execution stub (gas estimate, stub=true)  |

### 3.4 React Frontend (Port 5000)

The React 19 frontend is a compiled single-page application served by a lightweight Node.js HTTP server (serve\_frontend.js). It communicates exclusively through the /api/\* proxy path so that all service requests are same-origin from the browser. The application includes a live network dashboard, wallet management, a mining/Energy Provider interface, a multi-chain subnet builder, and the documentation hub. The frontend is pre-built -- no npm install or build step is required.

### 3.5 Service Dependency Matrix

| Service          | Depends On         | Direction    | Protocol       |
|------------------|--------------------|--------------|----------------|
| Python AI Svc    | Go Consensus :8181 | optional fwd | HTTP/REST      |
| Python AI Svc    | Rust Core :6000    | optional fwd | HTTP/JSON      |
| Frontend :5000   | Python AI Svc      | required     | HTTP/REST + WS |
| Go Consensus     | libp2p peers       | required     | TCP / QUIC     |
| Rust Core        | (none)             | standalone   | HTTP (server)  |
| Energy Providers | Python AI Svc      | outbound     | HTTP/REST      |

## 4. Proof of Intelligence (Pol) Consensus

### 4.1 Algorithm Overview

Proof of Intelligence replaces the brute-force nonce search of Proof of Work with a structured AI evaluation task. When a new block is proposed, each registered validator agent evaluates the block using a trained neural network and assigns an intelligence score in  $[0, 1]$ . A block is accepted when the trust-weighted average of validator scores exceeds the network threshold (0.60) and the PBFT quorum has endorsed it.

The Pol algorithm proceeds in four phases per block: (1) Proposal -- the designated proposer collects transactions from the mempool and assembles a candidate block; (2) Evaluation -- every registered validator independently scores the block using its local model; (3) Aggregation -- scores are weighted by each validator's `trust_weight` and averaged; (4) Commit -- if the aggregate score exceeds 0.60 and PBFT pre-commit messages form a quorum, the block is finalised and appended to the chain.

### 4.2 Local Block Scoring Model

Each external validator node runs a 3-feature scoring model (implemented in `trisp_i_energy_provider.py`) that requires only NumPy -- no external ML framework:

| Feature                      | Weight | Formula                                   |
|------------------------------|--------|-------------------------------------------|
| <code>tx_count_score</code>  | 0.40   | $\min(1.0, \text{tx\_count} / 50)$        |
| <code>hash_difficulty</code> | 0.30   | leading hex zeros in block hash / 4       |
| <code>recency_score</code>   | 0.30   | $\exp(-\max(0, \text{age\_s} - 15) / 30)$ |

$\text{score} = 0.40 * \text{tx\_count\_score} + 0.30 * \text{hash\_difficulty} + 0.30 * \text{recency\_score}$ , clipped to  $[0.0, 1.0]$ . The server's internal model (`ValidatorAgent`) uses an 8-input, 16-hidden-unit, 3-class MLP trained on historical block data.

### 4.3 Validator Registry & Trust Weights

Any external node may register as a Pol validator via `POST /api/validators/register`, providing its `provider_id`, Ed25519 public key (`pubkey_hex`), and TRP stake address. The registry maintains a `trust_weight` for each validator, initialised at 0.50 and updated after every scored block:

| Outcome         | Update Rule                                 | Effect                                      |
|-----------------|---------------------------------------------|---------------------------------------------|
| Honest score    | $\text{trust\_weight} *= 1.02$ (cap 1.00)   | Deviation $\leq 0.25$ from consensus median |
| Dishonest score | $\text{trust\_weight} *= 0.95$ (floor 0.10) | Deviation $> 0.25$ from consensus median    |

Validators whose `trust_weight` drops below 0.30 are excluded from consensus scoring and earn no TRP block rewards. The exclusion threshold provides Sybil resistance: an attacker registering many cheap nodes cannot influence consensus because their trust weights decay to 0.10 after just a few dishonest rounds. Recovery is possible by submitting honest scores that raise the weight back above 0.30 through the +2% honest bonus.

TRUST\_DECAY: 0.05 (5% penalty per dishonest verdict)

Trust floor: 0.10 (minimum; node still exists but earns nothing)

Trust ceiling: 1.00 (maximum; 2% bonus per honest verdict)

Exclusion threshold: 0.30 (below this: no consensus participation)

## 4.4 Energy Provider Earning Loops

Energy Providers (external nodes running `trispi_energy_provider.py`) earn TRP through three independent parallel loops. All rewards are issued as signed TRP transactions to the provider's registered stake address.

| Loop              | Reward   | Interval | Mechanism                                                                  |
|-------------------|----------|----------|----------------------------------------------------------------------------|
| Pol Block Scoring | 0.1 TRP  | ~15 s    | Score new block -> submit via <code>/api/validators/submit-score</code>    |
| FL Gradient       | 1.0 TRP  | ~30 s    | Compute gradient -> submit via <code>/api/federated/submit-gradient</code> |
| TX Fraud Verdict  | 0.01 TRP | ~5 s     | Score transaction -> submit signed verdict                                 |

The Pol reward is scaled by `trust_weight`:  $\text{reward} = 0.1 * \text{trust\_weight}$  (TRP). A validator at full trust (1.0) earns 0.1 TRP per block. A validator at minimum trust (0.10) earns 0.01 TRP per block. This creates a progressive incentive for maintaining accurate, honest scoring behaviour.

## 4.5 Fraud Detection Model v1

Every transaction submitted to the mempool is evaluated by a 5-feature fraud detection model. External validators run the identical model locally and submit signed verdicts. The consensus threshold is 0.65 -- a transaction classified as fraudulent by the majority of weighted validators is rejected from the block.

| Feature                        | Weight | Description                                                     |
|--------------------------------|--------|-----------------------------------------------------------------|
| <code>amount_anomaly</code>    | 0.30   | Z-score deviation of amount from sender historical mean         |
| <code>address_age_score</code> | 0.25   | New addresses are higher risk ( <code>tx_count &lt; 50</code> ) |
| <code>velocity_score</code>    | 0.20   | Rapid successive transactions from same address                 |
| <code>pattern_score</code>     | 0.15   | Round-number amounts, high-value round sums (bot signature)     |
| <code>graph_score</code>       | 0.10   | One-sided send/receive ratio (mule account signature)           |

$\text{fraud\_score} = \text{sum}(\text{feature\_i} * \text{weight\_i})$ , clipped to  $[0, 1]$ . If  $\text{fraud\_score} \geq 0.65$ , the transaction is flagged. The verdict is submitted with an Ed25519 signature over `sha3_256(tx_hash | verdict | provider_id)` to prevent impersonation or replay attacks.

## 4.6 Decentralised TX Validation Flow

Transaction validation proceeds in five steps, distributing the fraud-detection workload across all registered validators:

- \* Step 1: Client submits transaction to `POST /api/transactions/send`.
- \* Step 2: The Python AI service records the transaction in the mempool and broadcasts the `tx_hash` + metadata to all registered external validators.
- \* Step 3: Each validator runs the local 5-feature fraud model and submits a signed verdict (`fraud_score`, `verdict`) within 5 seconds.
- \* Step 4: The AI service collects verdicts, weights each by the validator's `trust_weight`, and computes the weighted consensus `fraud_score`.
- \* Step 5: If consensus `fraud_score`  $\geq 0.65$ , the transaction is rejected and the submitting address is flagged. Otherwise, the transaction enters the mempool and each honest validator earns 0.01 TRP.

## 4.7 Network Agent Implementation

The Python AI service runs two built-in network participation agents (`autonomous_agents.py`) that act as the server's own validator and compute provider identities on the network. These agents operate identically



to external Energy Providers, ensuring that the server always contributes to consensus even before any external validators connect.

### **ValidatorAgent**

Polls the Go consensus node for new blocks every 15 seconds. When a block is detected whose height exceeds the last processed height, the agent: (1) fetches the full block from `/api/blocks/{height}`; (2) scores it with the internal 8-feature MLP (TRISPINeuralNet); (3) submits the signed score via the internal consensus bus; (4) calls `dispatch_validator_reward()` for any external validators whose scores were classified as honest. The agent also runs `finalize_pending()` on each cycle to retry reward transactions that previously failed due to Go node connectivity issues.

### **ComputeProviderAgent**

Polls the FL round status every 30 seconds. When a new open round is detected, the agent: (1) fetches recent transactions from Go as training data; (2) runs one step of mini-batch SGD on the 2-layer MLP; (3) builds an Ed25519 signature over `sha3_256(provider_id|round_id|gradient_hash|stake_addr)`; (4) submits the encrypted gradient via `POST /api/federated/submit-gradient`. After aggregation, the ComputeProviderAgent's gradient is committed on-chain as part of the round commitment hash alongside all external provider gradients.

### **BlockScoreCollector**

Maintains an in-memory set of (block\_hash, provider\_id) pairs that have already been rewarded, ensuring idempotency of reward dispatch even if the ValidatorAgent retries failed transactions on subsequent cycles. The `was_rewarded()` / `mark_rewarded()` interface is used by `dispatch_validator_reward()` to prevent double-payment without requiring a database lookup.

## **4.8 Epoch Structure and Finality**

An epoch is a fixed window of 100 blocks (approximately 25 minutes at the 15-second block time). At epoch boundaries the network performs: Federated Learning v2 aggregation and on-chain commitment, validator reward distribution, trust weight recalculation, and EIP-1559 gas base fee recalibration. Block finality is deterministic -- once a block receives PBFT commit messages from a quorum of 13 of the 21 active validators it is irreversible.

Block time: 15 seconds

Epoch length: 100 blocks (~25 min)

Validator set size: 21 nodes (max\_validators in genesis.json)

PBFT quorum: 13 of 21 ( $f = 6$  Byzantine fault tolerance)

Min PoI score: 0.60

Fraud threshold: 0.65

Min validator stake: 10,000 TRP

Trust init value: 0.50

## 5. Federated Learning v2

---

### 5.1 Architecture Overview

TRISPI implements a federated learning protocol to train the shared fraud-detection model without requiring validators to share raw transaction data. In v3.0, the protocol is upgraded with end-to-end encryption, on-chain commitment, and Kyber1024-based key exchange, constituting Federated Learning v2.

The FL coordinator (`federated_learning_v2.py`) manages rounds. Each round proceeds as follows: (1) Announce -- coordinator opens a new round (`round_id`, `status='open'`); (2) Registration -- providers that want to participate call `POST /api/federated/register`; (3) Submission -- each provider trains locally and calls `POST /api/federated/submit-gradient` with an encrypted gradient payload; (4) Aggregation -- coordinator decrypts, validates, and FedAvg-aggregates all submitted gradients; (5) Commitment -- the SHA3-256 hash of the aggregated weights is posted as a signed on-chain transaction; (6) Reward -- each provider whose gradient was accepted into the round receives 1.0 TRP.

### 5.2 AES-256-GCM Gradient Encryption

Each gradient submission is encrypted with AES-256-GCM. The per-provider, per-round symmetric key is derived deterministically so no prior key exchange is required:

```
round_nonce = sha3_256(f"{round_id}|{provider_id}") # 32 bytes
aes_key     = sha3_256(provider_pubkey_bytes || round_nonce) # 32 bytes
nonce      = os.urandom(12) # 12 bytes GCM nonce
ciphertext = AESGCM(aes_key).encrypt(nonce, plaintext, None)
```

The server derives the same AES key from the registered `pubkey_hex` and `round_id`. No key transmission occurs over the wire. The gradient payload submitted to `POST /api/federated/submit-gradient` contains: `provider_id`, `signature_hex`, and `gradient = {ciphertext, nonce, encrypted: true}`.

The gradient itself is a one-step mini-batch SGD update on a 2-layer MLP (Input 4 -> Hidden 8 -> Output 2, cross-entropy loss, `lr = 0.01`). Architecture: `W1 (4x8)`, `b1 (8,)`, `W2 (8x2)`, `b2 (2,)`. Training labels are derived from the provider's local transaction data; providers with GPU (PyTorch + CUDA/MPS) use the GPU path automatically.

### 5.3 Submission Signature

Every gradient submission is accompanied by an Ed25519 signature that binds the provider identity, round, and gradient hash together:

```
gradient_hash = sha3_256(json.dumps(gradient, sort_keys=True))
msg_str       = f"{provider_id}|{round_id}|{gradient_hash}|{stake_address}"
msg_bytes     = sha3_256(msg_str.encode()).digest()
signature     = Ed25519PrivateKey.sign(msg_bytes)
```

The server verifies the signature against the registered `pubkey_hex` before decrypting or processing the gradient. Rejected submissions are logged but do not count against the provider's `trust_weight`.

### 5.4 On-Chain Aggregation Commitment

After every FL aggregation round, the coordinator posts the aggregation result as a verifiable on-chain record. The commitment data structure:

```
{
  "event":          "FL_AGGREGATION_COMMITMENT",
  "round_id":       42,
  "aggregated_hash": "sha3_256(json.dumps(aggregated_weights, sort_keys=True))",
  "n_contributors": 7,
  "model_version":  15,
  "timestamp":      1748700000.0
}
```

This JSON is embedded in the data field of a TRP transaction from the server's service key address. Anyone can verify the round using GET /api/federated/verify-round/{round\_id}, which returns the stored commitment hash and the current global model hash for comparison.

## 5.5 Differential Privacy

The FL coordinator applies gradient clipping (L2 norm  $\leq 5.0$ ) and additive Gaussian noise (sigma = 0.1) to the aggregated gradient before applying it to the global model. This provides (epsilon, delta)-differential privacy guarantees. Individual provider data cannot be reconstructed from the aggregated gradient with high probability.

## 5.6 Round API

| Method | Path                                   | Description                                 |
|--------|----------------------------------------|---------------------------------------------|
| GET    | /api/federated/round-status            | Current round id and status (open/closed)   |
| GET    | /api/federated/global-model            | Aggregated model metadata (shape, norm)     |
| GET    | /api/federated/global-model-weights    | Raw model weights (numeric lists)           |
| POST   | /api/federated/register                | Register as FL participant                  |
| POST   | /api/federated/submit-gradient         | Submit encrypted gradient for current round |
| GET    | /api/federated/verify-round/{round_id} | Verify on-chain aggregation commitment      |

## 6. Post-Quantum Cryptography

### 6.1 Hybrid Signature Scheme -- Ed25519 + Dilithium3

TRISPI uses a hybrid digital signature scheme in which every block-signing operation produces two independent signatures: one classical Ed25519 signature and one Dilithium3 (ML-DSA) post-quantum signature. Both are required for a block to be considered valid by the network. Transaction signatures from external clients use Ed25519 only (Dilithium3 is applied at the block level).

| Parameter        | Ed25519             | Dilithium3 (ML-DSA Level 3)          |
|------------------|---------------------|--------------------------------------|
| Public key size  | 32 bytes            | 1,952 bytes                          |
| Private key size | 32 bytes            | 4,000 bytes                          |
| Signature size   | 64 bytes            | 3,293 bytes                          |
| Security level   | 128-bit classical   | ~128-bit post-quantum (NIST Level 3) |
| Python library   | cryptography.hazmat | dilithium-py                         |
| Rust library     | --                  | pqcrypto-dilithium 0.5 (~40x faster) |
| Standard         | RFC 8032            | NIST FIPS 204 (finalised Aug 2024)   |

### 6.2 Key Exchange -- Kyber1024

Node-to-node session establishment and FL gradient key derivation uses Kyber1024 (ML-KEM), standardised as NIST FIPS 203, providing NIST Level 5 post-quantum security. In FL v2, the provider's Ed25519 public key seed and round\_nonce are combined via SHA3-256 to derive the AES-256-GCM session key, making it infeasible for an observer to recover the gradient plaintext without the private key.

Kyber1024 parameters: public key 1,568 bytes, ciphertext 1,568 bytes, shared secret 32 bytes.

### 6.3 NIST PQC Standards Compliance

| Standard | Algorithm           | Status in TRISPI v3.0                            |
|----------|---------------------|--------------------------------------------------|
| FIPS 203 | ML-KEM (Kyber1024)  | Implemented (kyber-py + key derivation in FL v2) |
| FIPS 204 | ML-DSA (Dilithium3) | Implemented (dilithium-py + pqcrypto-dilithium)  |
| FIPS 205 | SLH-DSA (SPHINCS+)  | Planned -- roadmap for long-lived root keys      |

### 6.4 Wallet Derivation

TRP wallet address derivation pipeline:

```
mnemonic -> PBKDF2-HMAC-SHA512(password='', iterations=2048) -> seed (64 B)
seed[:32] -> Ed25519PrivateKey(cryptography.hazmat.primitives)
ed25519_pubkey -> SHA-256 -> RIPEMD-160 -> Base58Check -> 'trp1...' address
seed[:32] -> Dilithium3.keygen() -> (pq_pk, pq_sk) (dilithium-py)
```

Energy Provider key derivation (non-wallet, persistent):

```
Ed25519PrivateKey.generate() -> saved to ~/.trispi/provider_key.json
TRP address = 'trp1' + sha3_256(pubkey_bytes).hexdigest()[:40]
```

## 7. Dual Runtime -- EVM and WASM

---

### 7.1 EVM Compatibility Layer

TRISPI's EVM contract runtime is implemented in the Python AI service (contract\_engine.py and real\_dual\_runtime.py) with an in-house EVM interpreter and a py-evm fallback. Each deployed contract is immediately audited by contract\_auditor.py. Contracts with risk\_score  $\geq 0.8$  are blocked from activation. Chain ID: 7878.

EVM deployment request body (POST /api/contracts/deploy/evm):

```
{
  "bytecode": "0x608060...",      // compiled Solidity bytecode
  "deployer": "trp1...",          // deployer TRP address
  "gas_limit": 30000000,           // maximum gas units
  "value": 0                       // TRP value transferred on deploy
}
```

### 7.2 WASM / CosmWasm Runtime

CosmWasm modules are deployed via POST /api/contracts/deploy/wasm. The WASM runtime validates module structure, allocates a sandbox, and returns a deterministic contract address. The Rust core bridge provides thin WASM execution stubs (POST /wasm/exec) as the integration target for the planned native runtime expansion.

### 7.3 EIP-1559 Gas Market

TRISPI implements the EIP-1559 fee mechanism with a dynamic base fee that adjusts each block to target 50% block utilisation. Each transaction must specify max\_fee\_per\_gas  $\geq$  base\_fee + priority\_fee. The base fee component is burned; the priority fee goes to the block proposer.

Gas target per block: 15,000,000 units

Gas limit per block: 30,000,000 units

Base fee initial: 1,000 gwei

Base fee adjustment: +/- 12.5% per block depending on utilisation

Burn fraction: 70% of base fee is burned

Proposer fraction: 30% of base fee + 100% of priority fee

## 8. P2P Network Layer

---

### 8.1 libp2p Transport

TRISPI uses libp2p for peer-to-peer communication. The Go consensus node manages all P2P traffic on port 50052. Each node maintains a routing table of known peers and uses a gossipsub protocol for block and transaction propagation. Maximum active peers: 50; minimum maintained: 8.

### 8.2 PBFT Consensus Protocol

The Go consensus node implements Practical Byzantine Fault Tolerance (PBFT) for block finality. A committee of up to 21 validators is selected each epoch based on trust\_weight and stake. The three PBFT phases are:

#### Pre-prepare

The designated leader broadcasts the block proposal with an incremented view number to all committee members.

#### Prepare

Each committee member validates the block proposal and broadcasts a signed PREPARE message. A node moves to commit phase when it receives  $2f+1 = 13$  PREPARE messages.

#### Commit

Each node broadcasts a signed COMMIT message. When a node receives 13 COMMIT messages it finalises the block.

### 8.3 Peer Scoring

The PeerScoreManager in peer\_scoring.go assigns a floating-point reputation score to each connected peer. Adjustment rules:

ScoreGoodMsg = +0.1: Valid, well-formed PBFT message received.

ScoreBadMsg = -2.0: Malformed or protocol-invalid message received.

ScoreInvalidBlock = -20.0: Peer submitted invalid block or equivocated.

ScoreBanThreshold = -10.0: Peer is automatically banned for 30 minutes.

Rate limit: 60 messages per minute per peer (anti-flood).

### 8.4 Bootstrap and Discovery

New nodes call GET /api/p2p/bootstrap on any known TRISPI node to receive the current peer list, chain head hash and height, and genesis parameters. Nodes self-announce via POST /api/p2p/announce with their libp2p multiaddr, Ed25519 public key, and current block height.

## 9. IBC Multi-Chain and Parachain Registry

---

### 9.1 Overview

TRISPI implements a lightweight IBC-compatible protocol that allows child chains (parachains) to register with the TRISPI hub, open cross-chain payment channels, and relay value through cryptographically verified Merkle Patricia Trie proofs. All parachain state is mirrored in the on-chain ChainRegistry contract.

### 9.2 Parachain Registration

A new parachain registers by calling POST /api/chains/register with its chain\_id, genesis\_hash, validator\_set, and IBC relay endpoint. The hub stores the registration and assigns a parachain slot. The parachain operator then configures their node to relay block headers to the hub at each epoch.

| Method | Path                         | Description                            |
|--------|------------------------------|----------------------------------------|
| POST   | /api/chains/register         | Register a new parachain with the hub  |
| GET    | /api/chains                  | List all registered parachains         |
| GET    | /api/chains/{chain_id}       | Get parachain details and status       |
| POST   | /api/chains/{chain_id}/relay | Relay a cross-chain packet             |
| GET    | /api/chains/{chain_id}/proof | Get Merkle proof for cross-chain state |

### 9.3 Cross-Chain Asset Transfer

Cross-chain TRP transfers use a lock-and-mint model: (1) Sender locks TRP on the source chain via POST /api/chains/lock-transfer; (2) A relay node submits the Merkle proof to the destination chain; (3) The destination chain verifies the proof against the source chain's committed block header and mints wrapped TRP to the recipient address.

## 10. TRP Token Economics

---

### 10.1 Token Parameters

Token symbol: TRP

Total supply (hard cap): 50,000,000 TRP

Chain ID: 7878

Initial block reward: 50 TRP

Halving interval: 210,000 blocks (~3.65 years)

Base fee burn: 70% of base fee is permanently destroyed

Validator share: 30% of base fee + 100% priority fee -> proposer

Energy Provider reward: 0.1 TRP/block + 1.0 TRP/FL round + 0.01 TRP/verdict

### 10.2 EIP-1559 Fee Mechanics

Block fees are split between burn (70%) and the block proposer (30% + priority fees). The base fee adjusts +/- 12.5% each block based on whether gas usage was above or below the 15M gas target. This creates a self-regulating fee market that smooths out demand spikes without manual governance intervention.

### 10.3 Halving Schedule

Block reward schedule (approximate, subject to halving at every 210,000 blocks):

| Epoch       | Blocks            | Block Reward | Cumulative Supply        |
|-------------|-------------------|--------------|--------------------------|
| 1 (Genesis) | 0 - 209,999       | 50 TRP       | ~10.5M TRP               |
| 2           | 210,000 - 419,999 | 25 TRP       | ~15.75M TRP              |
| 3           | 420,000 - 629,999 | 12.5 TRP     | ~18.375M TRP             |
| 4           | 630,000 - 839,999 | 6.25 TRP     | ~19.69M TRP              |
| ...         | ...               | ...          | ...                      |
| Asymptote   | -> infinity       | ~0 TRP       | ~21M TRP (burn adjusted) |

With 70% base fee burn, the effective circulating supply is significantly lower than the hard cap. Under moderate network activity, burn may offset new issuance making TRP deflationary from epoch 3 onwards.

### 10.4 Staking and Delegation

Validators must stake a minimum of 10,000 TRP to participate in PBFT consensus. Delegators can assign stake to validators via `POST /api/staking/delegate`. Delegated stake counts toward the validator's total weight in PBFT committee selection. Slashing events (double-sign: 5% of stake) apply to both validator and delegator stake proportionally.



## 11. Standalone Energy Provider

---

### 11.1 Overview

trisp.energy\_provider.py is a self-contained Python 3.10+ script that allows any internet-connected machine to participate in the TRISPI network and earn TRP tokens. No blockchain node, special hardware, or large stake deposit is required. The script handles all cryptographic setup, registration, and reward tracking automatically.

Minimum requirements:

- \* Python 3.10+ with pip
- \* Internet connection to a running TRISPI node
- \* Dependencies: httpx, numpy, cryptography (pip install httpx numpy cryptography)
- \* Optional: torch (for GPU acceleration on CUDA/Apple MPS hardware)

### 11.2 Getting Started

```
# Install dependencies
pip install httpx numpy cryptography

# Run against the public TRISPI node
python3 trisp.energy_provider.py --node https://your-trispi-node.example.com

# Or connect to a local node
python3 trisp.energy_provider.py --node http://localhost:8000

# Specify a custom provider ID
python3 trisp.energy_provider.py --node http://localhost:8000 --id my-provider

# Show full options
python3 trisp.energy_provider.py --help
```

On first run, the script generates a fresh Ed25519 keypair and saves it to ~/.trisp/provider\_key.json (permissions 600). The TRP address is derived as 'trp1' + first 40 hex chars of sha3\_256(pubkey\_bytes). IMPORTANT: back up this file -- it is the private key that controls your earned TRP.

### 11.3 Three Earning Loops

#### Loop 1 -- PoI Block Scoring (every ~15 seconds)

Polls GET /api/network/stats every 15 seconds. When a new block is detected, downloads the block header and evaluates it using the local 3-feature PoI model. Submits the score via POST /api/validators/submit-score with an Ed25519 signature. Earns 0.1 TRP \* trust\_weight per accepted score. Trust weight starts at 0.50 and adjusts +2% / -5% per round.

#### Loop 2 -- Federated Learning Gradient (every ~30 seconds)

Polls GET /api/federated/round-status every 30 seconds. When a new round opens, fetches recent transactions as training data, runs one SGD step on the local 2-layer MLP (Input 4 -> Hidden 8 -> Output 2), encrypts the gradient with AES-256-GCM, and submits it via POST /api/federated/submit-gradient.

Earns 1.0 TRP per accepted gradient. GPU providers use PyTorch if available.

### Loop 3 -- Transaction Fraud Verdict (every ~5 seconds)

Polls GET /api/transactions/pending every 5 seconds. For each unseen transaction, runs the local 5-feature fraud detection model and submits a signed verdict via POST /api/transactions/verdict. Earns 0.01 TRP per honest accepted verdict. Seen transaction hashes are cached in memory to avoid duplicate submissions.

## 11.4 Live Dashboard

The script displays a real-time terminal dashboard that updates every 5 seconds:

```
=====
TRISPI Energy Provider  --  Connected to http://node:8000
=====

Provider ID   : my-provider-abc123
TRP Address   : trp1a3f9d2e...
Uptime        : 01:23:45
CPU Usage     : 34.2%   RAM: 128.4 MB   GPU: RTX 4090

[PoI] Blocks scored:  127 | Accepted: 124 | Earned: 12.40 TRP
[FL]  Rounds entered:  18 | Accepted: 17 | Earned: 17.00 TRP
[TX]  Verdicts sent: 2341 | Accepted: 2308 | Earned: 23.08 TRP

TOTAL TRP EARNED: 52.48 TRP
Trust weight     : 0.8416
=====

Last block : #15729 | Peers: 12 | Connected: YES
=====
```

## 11.5 Command-Line Reference

Full list of command-line arguments for trispi\_energy\_provider.py:

| Argument         | Default                    | Description                                       |
|------------------|----------------------------|---------------------------------------------------|
| --node URL       | http://localhost:8000      | TRISPI node base URL                              |
| --id PROVIDER_ID | auto-generated UUID        | Unique provider identifier                        |
| --key-file PATH  | ~/trispi/provider_key.json | Ed25519 keypair file                              |
| --poi-interval S | 15                         | Seconds between PoI block score submissions       |
| --fl-interval S  | 30                         | Seconds between FL round checks                   |
| --tx-interval S  | 5                          | Seconds between TX fraud verdict polls            |
| --no-gpu         | off                        | Disable PyTorch GPU acceleration (use numpy only) |
| --no-color       | off                        | Disable ANSI colour in terminal dashboard         |
| --quiet          | off                        | Suppress dashboard, only log errors to stderr     |

## 11.6 Error Recovery

The script handles transient connectivity failures gracefully. Each loop operates independently -- a failure in one loop does not stop the others. Specific recovery behaviour:

### Node unreachable

Loops retry with exponential back-off starting at 5 s, capping at 60 s. The dashboard shows 'Connected:

NO' until the node responds.

#### **Gradient rejected**

If the server returns {accepted: false}, the round\_id is still marked as seen to prevent duplicate submission. The rejection reason is logged and displayed in the dashboard.

#### **Trust weight drops to 0.10**

The script continues submitting scores normally. Trust weight recovery is automatic: consecutive honest scores recover +2% per round. Exclusion (<0.30) is also recovered automatically once honest scores are submitted.

#### **Key file missing**

If ~/.trispi/provider\_key.json is deleted, a new keypair is generated on next startup. All earned TRP associated with the old key is permanently inaccessible -- backup this file.

#### **Duplicate block scored**

Block heights are tracked in memory. If the same block height is seen again after a restart, the score is skipped to avoid the server duplicate-rejection error.

### **11.7 Security Model**

The private key never leaves the local machine. All network communication uses Ed25519 signatures to authenticate every submission. The AES-256-GCM encryption ensures that gradient data cannot be read by third parties in transit. The trust weight mechanism ensures that a malicious provider cannot influence consensus beyond their proportional stake in honest behaviour. The server verifies every signature before processing any submission.

## 12. Security Model

---

### 12.1 Byzantine Fault Tolerance

TRISPI tolerates up to  $f = 6$  Byzantine (actively malicious) validators out of 21 in the PBFT committee, requiring  $2f+1 = 13$  honest nodes to make progress. The trust weight system provides an additional layer: even if 6 validators are Byzantine, their influence is further reduced by their low trust weights (dishonest validators lose 5% trust per round, reaching exclusion at 0.30 after approximately 25 consecutive dishonest rounds from the initial 0.50).

### 12.2 Sybil Resistance

Sybil attacks (creating many fake validator identities) are mitigated by the trust weight initialisation at 0.50 and the 10,000 TRP minimum stake requirement for full PBFT participation. New providers can submit PoI scores and FL gradients without stake, but their low initial trust weight (0.50) limits their influence. The cost of accumulating enough dishonest stake to influence consensus is prohibitive: an attacker would need to register and maintain enough nodes to control 6 of the 21 PBFT slots, requiring 60,000 TRP in stake.

### 12.3 Post-Quantum Threat Model

The hybrid Ed25519 + Dilithium3 signature scheme ensures security under both classical and quantum adversaries. An attacker with a quantum computer running Shor's algorithm could break Ed25519 but not Dilithium3. An attacker without quantum advantage cannot break either. The network will migrate to pure Dilithium3 (removing Ed25519) via a governance vote when quantum hardware reaches a threat threshold as defined by NIST guidance.

### 12.4 Replay Attack Prevention

Every signed message includes a nonce, round\_id, or block\_height component to prevent replay attacks. Transaction signatures include the chain\_id (7878) in the signed payload (EIP-155 compatible). PBFT messages include the view\_number and sequence\_number so old messages cannot be replayed in future PBFT rounds.

### 12.5 AI Security Middleware

The Python AI service applies AISecurityMiddleware to every incoming request. This middleware maintains an IP trust score for each requesting address, decremented on malformed requests and automatically banning IPs that drop below the configured threshold. Rate limiting (sliding window, 100 requests per minute per IP by default) prevents denial-of-service via request flooding.

### 12.6 Economic Security Analysis

TRISPI's economic security guarantees are analysed under three adversary models:

#### Rational adversary (profit-seeking)

A rational adversary controls  $f < 7$  validators and attempts to manipulate PoI scores to favour their own blocks. The trust weight mechanism limits their influence: after 10 dishonest rounds from the initial 0.50, trust drops to  $0.50 * 0.95^{10} = 0.299$ , hitting the exclusion threshold. The adversary must continuously

register new identities, each requiring 10,000 TRP stake, making sustained manipulation prohibitively expensive relative to block rewards.

### **Byzantine adversary (arbitrary behaviour)**

A Byzantine adversary controlling  $f=6$  validators can attempt to stall consensus by withholding PBFT votes. PBFT tolerates this: the remaining 15 honest validators exceed the  $2f+1=13$  quorum required to make progress. Block production continues; the Byzantine validators earn no block rewards and their stake is subject to slashing after the equivocation is detected.

### **Long-range attack**

An attacker who obtains old private keys cannot rewrite history because PBFT finality is immediate and irreversible: once 13 validators commit a block, no valid fork can exist at that height. Combined with the Dilithium3 post-quantum signatures, even a quantum adversary cannot forge the commitment signatures needed to create an alternative chain.

## **12.7 Contract Audit**

Every EVM contract deployment is automatically audited by `contract_auditor.py` before activation. The auditor checks for common vulnerability patterns (reentrancy, integer overflow, unchecked external calls) and returns a `risk_score` in  $[0, 1]$ . Contracts with `risk_score`  $\geq 0.8$  are blocked from activation. The audit result is stored with the contract record and returned in the deployment API response.

## 13. Current Challenges and Solutions

---

Every production blockchain faces engineering challenges that must be addressed before and during mainnet launch. This section documents the known limitations of TRISPI v3.0 and the concrete technical solutions either already implemented or scheduled on the roadmap.

### 13.1 Cold-Start: No External Validators at Launch

#### Challenge:

At network genesis, there are no external Energy Providers registered. The PoI consensus and FL rounds require at least one participant to function. Without validators, block scores cannot be aggregated and FL rounds cannot open, stalling the entire network.

#### Solution:

Two server-side autonomous agents (`ValidatorAgent` and `ComputeProviderAgent` in `autonomous_agents.py`) are started automatically with the Python AI service. They register as internal validators and compute providers, ensuring that consensus always has at least one honest participant regardless of external network size. Both agents operate identically to external Energy Providers -- they earn virtual TRP tracked internally and do not draw from the reward pool until external validators join. This guarantees liveness from block 0.

### 13.2 Slow Startup Time (~30 Seconds)

#### Challenge:

The Python AI service loads 1,055 genesis wallet balances from a JSON state file, initialises the TRISPINeuralNet model weights, runs the Dilithium3 library availability check, and starts the FL coordinator on startup. On modest hardware this takes approximately 30 seconds, during which the API is unavailable and the frontend shows a connection error.

#### Solution:

The `start_backend.sh` script launches the Go consensus node first (which is immediately ready) and the Python service second without a blocking `waitForPort` call. The frontend React app gracefully retries API calls every 3 seconds while showing a 'Connecting...' overlay. For production deployments, the genesis balance file is loaded once and cached; subsequent restarts skip the balance re-initialisation if the in-memory state is already populated. A planned optimisation (roadmap Q4 2026) will move balance loading to a background coroutine, reducing startup to under 5 seconds.

### 13.3 Trust Weight Gaming by Coordinated Validators

#### Challenge:

A group of colluding validators could deliberately agree on a false consensus median, making honest outlier validators appear dishonest and causing their trust weights to decay. If the attacker controls enough stake to dominate the initial round scores, they can gradually exclude honest validators from consensus.

#### Solution:

Three countermeasures are in place. First, the PBFT quorum of 13 of 21 provides a hard Byzantine fault limit independent of trust weights -- colluders must control more than  $f=6$  of the 21 committee slots,

requiring 70,000+ TRP in stake. Second, the trust weight floor of 0.10 ensures honest validators are never fully silenced -- their votes still carry 10% weight and they can recover by submitting scores that eventually re-align with the true median as new honest validators join. Third, the server's built-in ValidatorAgent always submits an independent score derived from the TRISPINeuralNet model, providing a trusted anchor that cannot be excluded by external colluders.

## 13.4 Federated Learning Gradient Poisoning

### Challenge:

A malicious Energy Provider could submit carefully crafted (poisoned) gradients designed to bias the global fraud detection model towards accepting fraudulent transactions or falsely flagging legitimate ones. In standard FedAvg, a single provider with a large data set could disproportionately influence the global model.

### Solution:

FL v2 applies three defences before aggregation. (1) Gradient norm clipping: any gradient with L2 norm  $> 5.0$  is normalised to exactly 5.0 before aggregation, preventing a single submission from dominating the update direction. (2) Additive Gaussian noise with  $\sigma = 0.1$  is added to the aggregated gradient, providing (epsilon, delta)-differential privacy and making it statistically impossible to reconstruct individual training data or reverse-engineer a poisoning strategy. (3) Every gradient submission requires a valid Ed25519 signature from the registered pubkey\_hex; anonymous submissions are rejected. The on-chain commitment hash makes the aggregation auditable post-hoc.

## 13.5 Go Consensus Node as Single Point of Failure

### Challenge:

The Go consensus node (trispici-consensus binary) runs as a single process on port 8181. If it crashes or becomes unreachable, PBFT cannot proceed and new blocks cannot be finalised, even though the Python AI service and all Energy Providers remain operational.

### Solution:

The Python AI service includes a standalone fallback mode: when the Go node is unreachable for more than 30 seconds, it switches to a single-validator mode where it produces and self-signs blocks without the PBFT quorum. These blocks are marked with a 'standalone' flag and are re-submitted to the PBFT pipeline when the Go node recovers, allowing seamless catchup without data loss. For production deployments, the roadmap includes a supervisor process (systemd or Kubernetes Pod restart policy) that automatically restarts the Go binary within 5 seconds of failure.

## 13.6 Mempool Spam and DoS Attacks

### Challenge:

A malicious actor could flood the mempool with low-value or zero-value transactions to consume validator compute resources (fraud scoring), slow block production, or exhaust the API rate limit for legitimate users. The current rate limit of 100 requests/minute per IP is bypassable from distributed source IPs.

### Solution:

Three layers of protection are active. First, AISecurityMiddleware assigns an IP trust score that decays on repeated malformed requests and auto-bans IPs that fall below the ban threshold, blocking the source

at the connection level without consuming handler resources. Second, the fraud detection model runs on every mempool submission and automatically rejects transactions with `fraud_score >= 0.65` -- spam bursts from unknown addresses trigger the `address_age_score` and `velocity_score` features, pushing them above the threshold. Third, a minimum base fee (EIP-1559 floor: 1,000 gwei) makes sustained mempool flooding economically costly relative to the attacker's benefit.



## 14. Conclusion and Roadmap

---

### 14.1 Summary

TRISPI v3.0 delivers a functionally complete Web4 blockchain with AI-native consensus, post-quantum cryptography, decentralised machine learning, and an open participation model that allows any machine with an internet connection to contribute compute and earn TRP tokens. The three-stream Energy Provider reward model (PoI scoring, FL gradients, TX verdicts) creates genuine economic incentives for honest validator behaviour, replacing proof-of-work's wasteful energy expenditure with AI work that improves the network's own security.

### 14.2 Roadmap

#### Q3 2026 -- Validator Governance

On-chain governance voting for protocol parameter changes (gas targets, trust thresholds, epoch length) via TRP-weighted proposals.

#### Q4 2026 -- Native EVM Runtime in Rust

Replace the Python py-evm fallback with a native Rust EVM runtime integrated directly into the Rust core bridge, achieving 10-100x gas execution speed.

#### Q1 2027 -- SPHINCS+ Integration (FIPS 205)

Add SLH-DSA long-lived root keys for genesis validator identities, completing the full NIST post-quantum suite.

#### Q2 2027 -- Mainnet Launch

Public mainnet launch with the genesis validator set, initial TRP distribution, and open Energy Provider participation. Target: 500 active providers at launch.

#### Q3 2027 -- Cross-Chain Bridge v2

Upgrade the IBC relay to support atomic swaps and liquidity pools across registered parachains without a trusted intermediary.

#### 2028 -- Pure PQC Migration

Remove Ed25519 from the hybrid scheme after NIST quantum threat assessment recommends migration. Full Dilithium3-only signature protocol activated via governance vote.

## Appendix A. Full API Reference (Python AI Service :8000)

### Network & Health

| Method | Path               | Description                                  |
|--------|--------------------|----------------------------------------------|
| GET    | /api/network/stats | Network stats: height, peers, TPS, Pol score |
| GET    | /api/health        | Service health check                         |
| GET    | /api/genesis       | Genesis parameters                           |
| GET    | /api/ppc/status    | PQC library availability and mode            |

### Blocks & Transactions

| Method | Path                      | Description                            |
|--------|---------------------------|----------------------------------------|
| GET    | /api/blocks/latest        | Latest block header                    |
| GET    | /api/blocks/{height}      | Block at specific height               |
| GET    | /api/transactions/pending | Pending mempool transactions           |
| POST   | /api/transactions/send    | Submit a signed transaction            |
| POST   | /api/transactions/verdict | Submit fraud verdict for a transaction |

### Validator Registry (v3.0)

| Method | Path                         | Description                                      |
|--------|------------------------------|--------------------------------------------------|
| POST   | /api/validators/register     | Register Ed25519 pubkey as Pol validator         |
| POST   | /api/validators/submit-score | Submit signed block Pol score                    |
| GET    | /api/validators/list         | List all registered validators and trust weights |
| GET    | /api/validators/{id}         | Get validator details                            |

### Federated Learning v2

| Method | Path                                | Description                            |
|--------|-------------------------------------|----------------------------------------|
| GET    | /api/federated/round-status         | Current FL round id and status         |
| GET    | /api/federated/global-model         | Global model metadata                  |
| GET    | /api/federated/global-model-weights | Raw model weights (numeric lists)      |
| POST   | /api/federated/register             | Register as FL provider                |
| POST   | /api/federated/submit-gradient      | Submit AES-256-GCM encrypted gradient  |
| GET    | /api/federated/verify-round/{id}    | Verify on-chain aggregation commitment |

### Wallets & Staking

| Method | Path                          | Description                         |
|--------|-------------------------------|-------------------------------------|
| POST   | /api/wallet/create            | Create new TRP wallet from mnemonic |
| GET    | /api/wallet/{address}/balance | Get TRP balance                     |
| GET    | /api/wallet/{address}/txs     | Transaction history                 |
| POST   | /api/staking/stake            | Stake TRP as validator              |

|      |                              |                             |
|------|------------------------------|-----------------------------|
| POST | /api/staking/delegate        | Delegate stake to validator |
| GET  | /api/staking/slashing-events | Recent slashing events      |

## Smart Contracts

| Method | Path                       | Description                            |
|--------|----------------------------|----------------------------------------|
| POST   | /api/contracts/deploy/evm  | Deploy EVM (Solidity) contract         |
| POST   | /api/contracts/deploy/wasm | Deploy WASM (CosmWasm) contract        |
| POST   | /api/contracts/call        | Call a deployed contract               |
| GET    | /api/contracts/{addr}      | Get contract metadata and audit result |

## Multi-Chain (IBC)

| Method | Path                         | Description              |
|--------|------------------------------|--------------------------|
| POST   | /api/chains/register         | Register a new parachain |
| GET    | /api/chains                  | List all parachains      |
| GET    | /api/chains/{chain_id}       | Get parachain details    |
| POST   | /api/chains/{chain_id}/relay | Relay cross-chain packet |

## Energy Provider

| Method | Path                       | Description                          |
|--------|----------------------------|--------------------------------------|
| POST   | /api/ai-energy/register    | Register as Energy Provider (legacy) |
| POST   | /api/ai-energy/heartbeat   | Send compute stats heartbeat         |
| GET    | /api/ai-energy/leaderboard | Energy Provider earnings leaderboard |

## Go Consensus Node :8181

| Method | Path                      | Description                       |
|--------|---------------------------|-----------------------------------|
| GET    | /api/p2p/bootstrap        | Bootstrap: chain head + peer list |
| POST   | /api/p2p/announce         | Self-announce as peer             |
| GET    | /api/p2p/peers            | Current peer list                 |
| POST   | /api/p2p/pbft/pre-prepare | PBFT pre-prepare message handler  |
| POST   | /api/p2p/pbft/prepare     | PBFT prepare message handler      |
| POST   | /api/p2p/pbft/commit      | PBFT commit message handler       |

## Rust Core Bridge :6000

| Method | Path       | Description                      |
|--------|------------|----------------------------------|
| GET    | /health    | Health and Dilithium3 capability |
| GET    | /pubkey    | Node Dilithium3 public key (hex) |
| POST   | /sign      | Sign payload with Dilithium3     |
| POST   | /verify    | Verify Dilithium3 signature      |
| POST   | /evm/exec  | EVM execution stub               |
| POST   | /wasm/exec | WASM execution stub              |

## Appendix B. genesis.json Schema

---

The genesis.json file is read by the Python AI service at startup. GET /api/genesis returns its contents. Operator-configurable fields:

```
{
  "chain_id":          7878,
  "chain_name":        "TRISPI Mainnet",
  "block_time_s":      15,
  "epoch_blocks":      100,
  "gas_target":        15000000,
  "gas_limit":         30000000,
  "base_fee_initial":   1000,
  "base_fee_burn_pct":  0.70,
  "min_poi_score":      0.60,
  "fraud_threshold":    0.65,
  "max_validators":     21,
  "min_validator_stake": 10000,
  "trust_init":         0.50,
  "trust_honest_bonus": 0.02,
  "trust_dishonest_decay": 0.05,
  "trust_exclusion":     0.30,
  "slash_double_sign":  0.05,
  "pqc_mode":          "hybrid",
  "ibc_enabled":        true,
  "go_consensus_port":  8181,
  "go_p2p_port":        50052,
  "validator_set": [ ... ],
  "balances": [ ... ]
}
```

chain\_id: EIP-155 chain identifier (7878 = TRISPI mainnet)  
block\_time\_s: Target block production interval in seconds (15)  
epoch\_blocks: Blocks per epoch -- FL aggregation cycle (100)  
gas\_target: Target block gas usage for EIP-1559 base fee anchor  
base\_fee\_burn\_pct: Fraction of base fee destroyed per block (0.70 = 70%)  
min\_poi\_score: Minimum aggregate PoI score to accept a block  
fraud\_threshold: Minimum weighted consensus fraud\_score to reject tx  
trust\_honest\_bonus: Trust weight multiplier for honest PoI score (1.02)  
trust\_dishonest\_decay: Trust weight multiplier for dishonest score (0.95)  
trust\_exclusion: Trust weight below which validator is excluded (0.30)  
slash\_double\_sign: Fraction of stake burned for equivocation (0.05 = 5%)  
pqc\_mode: 'hybrid' (Ed25519+Dilithium3) or 'pqc\_only'  
go\_consensus\_port: HTTP port of the Go consensus node (8181)  
go\_p2p\_port: libp2p P2P port of the Go consensus node (50052)

## References

---

- [1] Castro, M. & Liskov, B. (1999). Practical Byzantine Fault Tolerance. OSDI 1999, pp. 173-186.
- [2] NIST (2024). FIPS 203 -- Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM / Kyber1024). National Institute of Standards and Technology.
- [3] NIST (2024). FIPS 204 -- Module-Lattice-Based Digital Signature Standard (ML-DSA / Dilithium3). National Institute of Standards and Technology.
- [4] NIST (2024). FIPS 205 -- Stateless Hash-Based Digital Signature Standard (SLH-DSA / SPHINCS+). National Institute of Standards and Technology.
- [5] Buterin, V. (2021). EIP-1559: Fee market change for ETH 1.0 chain. Ethereum Improvement Proposals.
- [6] Bernstein, D. J. (2006). Curve25519: New Diffie-Hellman speed records. PKC 2006, LNCS 3958, pp. 207-228.
- [7] Burdges, J., et al. (2020). Overview of Polkadot and its Design Considerations. arXiv:2005.13456.
- [8] Buchman, E., Kwon, J. & Milosevic, Z. (2018). The latest gossip on BFT consensus. arXiv:1807.04938.
- [9] Kwon, J. & Buchman, E. (2016). Cosmos: A Network of Distributed Ledgers. Cosmos Whitepaper.
- [10] libp2p Specification. <https://github.com/libp2p/specs>
- [11] CosmWasm Documentation. <https://book.cosmwasm.com>
- [12] McMahan, H. B., et al. (2017). Communication-Efficient Learning of Deep Networks from Decentralised Data (FedAvg). AISTATS 2017.
- [13] Ethereum Yellow Paper. Wood, G. (2014, updated 2024). Ethereum: A Secure Decentralised Generalised Transaction Ledger.
- [14] TRISPI Source Repository. <https://github.com/TRISPIAINETWORK/TRISPI>
- [15] RFC 8032 -- Edwards-Curve Digital Signature Algorithm (EdDSA). Josefsson, S. & Liusvaara, I. (2017). IETF.
- [16] pqcrypto-dilithium 0.5. Rust implementation of Dilithium3. <https://crates.io/crates/pqcrypto-dilithium>
- [17] FastAPI Framework. <https://fastapi.tiangolo.com>
- [18] dilithium-py: Pure-Python Dilithium implementation. <https://github.com/GiacomoPope/dilithium-py>
- [19] cryptography.hazmat: Python cryptography library. <https://cryptography.io/en/latest/>
- [20] AES-GCM NIST SP 800-38D. Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC.

## Appendix C. Glossary

---

### **AES-256-GCM**

Advanced Encryption Standard in Galois/Counter Mode with a 256-bit key. An authenticated encryption scheme that provides both confidentiality and integrity. Used in TRISPI FL v2 to encrypt gradient payloads in transit.

### **BFT (Byzantine Fault Tolerance)**

A property of a distributed system that allows it to continue operating correctly even when some nodes fail or act maliciously. TRISPI tolerates up to  $f=6$  Byzantine nodes in a committee of 21 (PBFT protocol).

### **Block Reward**

TRP tokens minted and awarded to the block proposer for each successfully finalised block. The initial reward is 50 TRP, halving every 210,000 blocks.

### **Chain ID**

A unique integer identifier for the TRISPI mainnet blockchain (7878). Included in all signed transactions to prevent cross-chain replay attacks (EIP-155 compatible).

### **ComputeProviderAgent**

The server-side autonomous agent that participates in FL rounds as the server's own compute identity, training a local MLP and submitting encrypted gradients to the FL coordinator.

### **Dilithium3 (ML-DSA)**

A module-lattice-based digital signature algorithm standardised as NIST FIPS 204. Provides post-quantum security at NIST Level 3 (~128-bit quantum resistance). Signature size: 3,293 bytes.

### **Ed25519**

An elliptic-curve digital signature algorithm based on Curve25519. Provides 128-bit classical security. Signature size: 64 bytes. Used for all transaction signatures and Energy Provider authentication in TRISPI.

### **Energy Provider**

Any node (machine) that runs `trisp_i_energy_provider.py` to contribute CPU/GPU compute resources to the network. Earns TRP through three parallel loops: PoI scoring, FL gradient submission, and TX fraud verdicts.

### **EIP-1559**

Ethereum Improvement Proposal 1559. A transaction fee mechanism that introduces a base fee (burned) and a priority fee (to proposer), replacing the simple first-price auction model.

### **FedAvg**

Federated Averaging. The aggregation algorithm used in FL v2: each provider's gradient is weighted by its validation data size, then the weighted mean is computed and applied to the global model.

### **Federated Learning (FL)**

A machine learning technique where multiple participants train a shared model on their local data without sharing the raw data. TRISPI FL v2 uses encrypted gradients and on-chain commitment hashes.

### **Fraud Score**

A float in  $[0, 1]$  produced by the 5-feature fraud detection model for each transaction. A score  $\geq 0.65$  (fraud\_threshold) causes the transaction to be rejected from the mempool by weighted consensus.

### **Gradient**

The partial derivative of a loss function with respect to model weights, used to update the model in the direction that minimises the loss. In FL v2, each provider computes a gradient locally and submits it encrypted.

### **IBC (Inter-Blockchain Communication)**

A protocol for relaying messages and value between independent blockchains. TRISPI implements a lightweight IBC-compatible relay that uses Merkle Patricia Trie proofs for cross-chain state verification.

### **Kyber1024 (ML-KEM)**

A module-lattice-based key encapsulation mechanism standardised as NIST FIPS 203. Provides NIST Level 5 post-quantum security. Used for deriving FL session keys in TRISPI v3.0.

### **libp2p**

A modular networking stack used for peer-to-peer communication. TRISPI's Go consensus node uses libp2p for peer discovery, gossipsub block propagation, and PBFT message transport on port 50052.

### **Mempool**

The in-memory pool of submitted transactions that have not yet been included in a finalised block. Transactions in the mempool are evaluated by the fraud detection pipeline before block inclusion.

### **PBFT (Practical Byzantine Fault Tolerance)**

A consensus protocol designed by Castro & Liskov (1999) that achieves agreement among  $n=3f+1$  nodes while tolerating  $f$  Byzantine faults. TRISPI uses PBFT with  $n=21$ ,  $f=6$ , requiring 13 votes for finality.

### **Pol (Proof of Intelligence)**

TRISPI's consensus mechanism. Validators score proposed blocks using a locally-run neural network. The trust-weighted average of scores must exceed 0.60 for a block to be accepted. Replaces wasteful proof-of-work.

### **TRP**

The native token of the TRISPI network. Hard cap: 50,000,000 TRP. Used for transaction fees (EIP-1559), validator staking, delegation, and Energy Provider rewards.

### **Trust Weight**

A float in  $[0.10, 1.00]$  assigned to each registered validator, reflecting the historical accuracy of their Pol scores relative to the consensus median. Initialised at 0.50; +2% per honest verdict, -5% per dishonest verdict.

### **ValidatorAgent**

The server-side autonomous agent that polls the Go node for new blocks, scores them using the internal TRISPINeuralNet, and dispatches TRP reward transactions to external validators whose scores were classified as honest.

### **ValidatorRegistry**

The in-memory (and optionally PostgreSQL-backed) registry of external Pol validators. Stores `pubkey_hex`, `stake_address`, `trust_weight`, and submission counts for each registered `provider_id`.

### **WASM (WebAssembly)**

A binary instruction format designed as a portable compilation target. TRISPI supports CosmWasm modules deployed via `POST /api/contracts/deploy/wasm`, enabling Rust/Go smart contracts to run alongside Solidity EVM contracts.